DS Homework 4

1. Implement a Stack using Queue data structure.

Code:

```c
#include <stdio.h>
int q1[100],q2[100];
int front1=0,front2=0,rear1=-1,rear2=-1;
void push(int n){
    q2[++rear2]=n;
    while(front1<=rear1){
        q2[++rear2]=q1[front1++];
    }
    rear1=-1;
    front1=0;
    while(front2<=rear2){
        q1[++rear1]=q2[front2++];
    }
    rear2=-1;
    front2=0;
}
int pop(){
    if(rear1==-1){
        printf("Stack underflow");
        return -1;
    }
    return q1[front1++];
}
int peek(){
    if(rear1==-1){
        printf("Stack underflow");
        return -1;
    }
    return q1[front1];
}
void display(){
    int i;
    for(i=front1;i<=rear1;i++){
        printf("%d ",q1[i]);
    }
}
int main(){
    push(10);
    push(20);
    push(30);
    display();
    printf("\nDeleted element: %d\n",pop());
    printf("Top element: %d\n",peek());
    display();
```

```
    return 0;
}
```

Output:

```
PS D:\ds_hw_4> gcc 1.c -o 1
PS D:\ds_hw_4> ./1
30 20 10
Deleted element: 30
Top element: 20
20 10
PS D:\ds_hw_4>
```

2.Implement a Queue using Stack data structure.
Code:

```c
#include <stdio.h>
int s1[100],s2[100];
int top1=-1,top2=-1;
void enqueue(int n){
    s2[++top2]=n;
    int i;
    for(i=0;i<=top1;i++){
        s2[++top2]=s1[i];
    }
    top1=-1;
    for(i=0;i<=top2;i++){
        s1[++top1]=s2[i];
    }
    top2=-1;
}
int dequeue(){
    if(top1==-1){
        printf("Stack underflow");
        return -1;
    }
    return s1[top1--];
}
int front(){
    if(top1==-1){
        printf("Stack underflow");
        return -1;
    }
    return s1[top1];
}
void display(){
    int i;
    for(i=0;i<=top1;i++){
```

```
        printf("%d ",s1[i]);
    }
}
int main(){
    enqueue(7);
    enqueue(10);
    enqueue(70);
    display();
    printf("\nDequeued element: %d\n",dequeue());
    printf("Front element: %d\n",front());
    display();
    return 0;
}
```

Output:

```
PS D:\ds_hw_4> gcc 2.c -o 2
PS D:\ds_hw_4> ./2
70 10 7
Dequeued element: 7
Front element: 10
70 10
PS D:\ds_hw_4>
```

3.Write a C program using stack data structure to implement managing the history of web pages visited.

Code:

```
#include <stdio.h>
#include <string.h>
char s[100][100];
int top=-1;
void push(){
    if(top==99){
        printf("Stack overflow");
        return;
    }
    char name[100];
    printf("Enter the page visited:");
    scanf("%s",name);
    strcpy(s[++top],name);
}
void pop(){
    if(top==-1){
        printf("Stack underflow");
        return ;
    }
```

```
    printf("Deleted page: %s",s[top--]);
}
void peek(){
    if(top==-1){
        printf("Stack underflow");
        return ;
    }
    printf("Latest page: %s",s[top]);
}
void display(){
    int i,j;
    for(i=0;i<=top;i++){
        printf("%s\n",s[i]);
    }
}
int main(){
    int n;
    while(1){
        printf("1.Visit webpage\n2.Remove latest\n3.Latest
page\n4.Display\n5.Exit\nEnter option:");
        scanf("%d",&n);
        if(n==5){
            break;
        }
        switch(n){
            case 1:
            push();
            break;
            case 2:
            pop();
            break;
            case 3:
            peek();
            break;
            case 4:
            display();
            break;
            default:
            printf("Invalid input");
            break;
        }
    }
    return 0;
}
```

Output:

```
PS D:\ds_hw_4> gcc 3.c -o 3
PS D:\ds_hw_4> ./3
1.Visit webpage
2.Remove latest
3.Latest page
4.Display
5.Exit
Enter option:1
Enter the page visited:instagram
1.Visit webpage
2.Remove latest
3.Latest page
4.Display
5.Exit
Enter option:1
Enter the page visited:google
1.Visit webpage
2.Remove latest
3.Latest page
4.Display
5.Exit
Enter option:2
Deleted page: google1.Visit webpage
2.Remove latest
3.Latest page
4.Display
5.Exit
Enter option:3
Latest page: instagram1.Visit webpage
2.Remove latest
3.Latest page
4.Display
5.Exit
Enter option:4
instagram
1.Visit webpage
2.Remove latest
3.Latest page
4.Display
5.Exit
Enter option:5
PS D:\ds_hw_4>
```

4.Write a C program to sort a stack using temporary stack.
Code:

```c
#include <stdio.h>
#define MAX 100
int stack[MAX], temp[MAX];
int top = -1, ttop = -1;
void push(int s[], int *top, int value)
{
    s[++(*top)] = value;
}
int pop(int s[], int *top)
{
    return s[(*top)--];
}
int main()
{
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &x);
        push(stack, &top, x);
    }
    while(top != -1)
    {
        x = pop(stack, &top);

        while(ttop != -1 && temp[ttop] > x)
        {
            push(stack, &top, pop(temp, &ttop));
        }

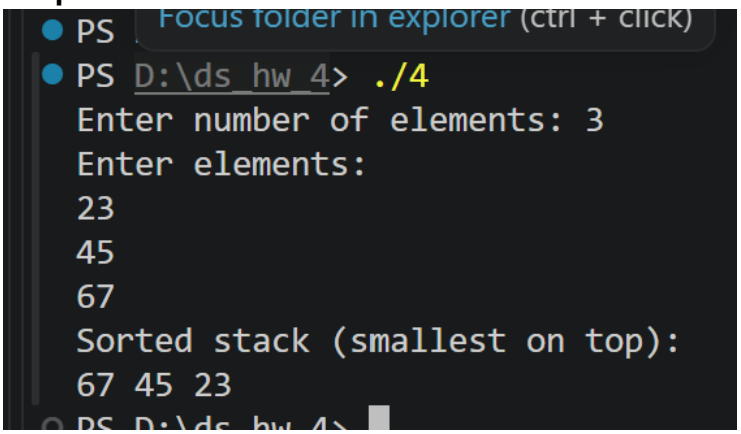
        push(temp, &ttop, x);
    }

    printf("Sorted stack (smallest on top):\n");

    while(ttop != -1)
    {
        printf("%d ", pop(temp, &ttop));
    }
return 0;
```

```
}
```

Output:



5.The Stock Span Problem is a financial problem where we have a series of daily price given for a stock denoted by an array and the task is to calculate the span of the stock's price for all days. The span of a stock's price for i'th day gives the maximum number of consecutive leading up to i'th day (including current day) where stock's price is less than or equal to its price on day i.
Example: I/p: [100, 80, 60, 70, 60, 75, 85] O/p: [1, 1, 1, 2, 1, 4, 6]
Code:

```c
#include <stdio.h>
#define MAX 100
int main()
{
    int price[MAX], span[MAX], stack[MAX];
    int n, top = -1;
    int i;

    printf("Enter number of days: ");
    scanf("%d", &n);

    printf("Enter prices:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &price[i]);
    for(i = 0; i < n; i++)
    {
        while(top != -1 && price[stack[top]] <= price[i])
            top--;

        if(top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];

        stack[++top] = i;
    }
    printf("Stock spans:\n");
    for(i = 0; i < n; i++)
```

```
        printf("%d ", span[i]);
    return 0;
}
```

Output:

```
PS D:\ds_hw_4> ./5
Enter number of days: 2
Enter prices:
500
600
Stock spans:
1 2
```

6.Given an array, for each element find the nearest greater element to its right that has a higher frequency than the current element. If no such element exists, set value to -1. Implement using stack. I/p: [2, 1, 1, 3, 2, 1] O/p: [1, -1, -1, 2, 1, -1]

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int a[MAX], freq[MAX] = {0};
    int ans[MAX], stack[MAX],n, top = -1,i, j;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < n; j++)
        {
            if(a[i] == a[j])
                freq[i]++;
        }
    }
    for(i = 0; i < n; i++)
        ans[i] = -1;
    for(i = 0; i < n; i++)
    {
        while(top != -1 &&
              freq[stack[top]] < freq[i])
        {
            ans[stack[top]] = a[i];
            top--;
```

```
        }

        stack[++top] = i;
    }

    printf("Output:\n");

    for(i = 0; i < n; i++)
        printf("%d ", ans[i]);

    return 0;
}
```

Output:

```
PS D:\ds_hw_4> gcc 6.c -o 6
PS D:\ds_hw_4> ./6
Enter number of elements: 3
Enter elements:
12
34
56
Output:
-1 -1 -1
PS D:\ds_hw_4>
```

7.Given a pattern containing only 1s and 2s. For increasing 1s and decreasing 2s, devise an algorithm using stack to print minimum digits required following that pattern. Digits range from 1-9 and digits can't repeat. I/p: 12 O/p: 213

Code:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char pattern[100];
    int stack[100];
    int top = -1;
    int i, n;
    printf("Enter pattern (I/D): ");
    scanf("%s", pattern);
    n = strlen(pattern);
    for(i = 0; i <= n; i++)
    {
        stack[++top] = i + 1;

        if(i == n || pattern[i] == 'I')
        {
            while(top != -1)
```

```
{
                printf("%d", stack[top--]);
            }
        }
    }

    return 0;
}
```

Output:

```
  -1 -1 -1
PS D:\ds_hw_4> gcc 7.c -o 7
PS D:\ds_hw_4> ./7
Enter pattern (I/D): I
12
PS D:\ds_hw_4> ./7
Enter pattern (I/D): D
21
PS D:\ds_hw_4>
```

8.Given an array of digits (values are from 0 to 9), find the maximum possible sum of two numbers formed from digits of the array. All digits of given array must be used to form the two numbers. I/p: [6, 8, 4, 5, 2, 3] O/p: 604 + (minimum sum formed by 3 digits) = 358 and 246

Code:

```c
#include <stdio.h>
int main()
{
    int a[100];
    int n, i, j, temp;
    int num1 = 0, num2 = 0;
    printf("Enter number of digits: ");
    scanf("%d", &n);
    printf("Enter digits:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for(i = 0; i < n - 1; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
```

```
    for(i = 0; i < n; i++)
    {
        if(i % 2 == 0)
            num1 = num1 * 10 + a[i];
        else
            num2 = num2 * 10 + a[i];
    }
    printf("Number 1 = %d\n", num1);
    printf("Number 2 = %d\n", num2);
    printf("Minimum sum = %d\n", num1 + num2);
    return 0;
}
```

Output:

```
PS D:\ds_hw_4> ./78
Enter number of digits: 2
Enter digits:
23
45
Number 1 = 23
Number 2 = 45
Minimum sum = 68
PS D:\ds_hw_4>
```

9.Given an array and a positive integer k, find the first negative integer for each window (continuous subarray) of size k. If a window does not contain a negative integer, print 0. Implement using queue. I/p: [-8, 2, 3, -6, 10] k = 2 O/p: [-8, 0, -6, -6]

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int a[MAX], queue[MAX];
    int n, k;
    int front = 0, rear = -1;
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter window size: ");
    scanf("%d", &k);

    for(i = 0; i < n; i++)
    {
```

```
        if(a[i] < 0)
            queue[++rear] = i;
        if(front <= rear && queue[front] <= i - k)
            front++;
        if(i >= k - 1)
        {
            if(front <= rear)
                printf("%d ", a[queue[front]]);
            else
                printf("0 ");
        }
    }

    return 0;
}
```

Output:

```
Minimum sum = 68
PS D:\ds_hw_4> gcc 9.c -o 9
PS D:\ds_hw_4> ./9
Enter number of elements: 3
Enter array:
23
45
90
Enter window size: 2
0 0
PS D:\ds_hw_4>
```

10.Given a matrix of dimension m x n where each cell in the matrix can have values 0, 1, or 2, which has the following meaning: 0: empty cell 1: cells having fresh oranges 2: cells having rotten oranges The task is to find the minimum time required so that all oranges become rotten. A rotten orange at index (i,j) can rot fresh oranges which are its neighbors (up, down, left and right). If impossible to rot every orange, return -1. Use Queue for implementation. Example: I/p: [[2,1,0,2,1], [1,0,1,2,1], [1,0,0,2,1]] O/p: 2

Code:

```
#include <stdio.h>
#define MAX 100
int main()
{
    int a[MAX][MAX];
    int queue[MAX * MAX][2];
    int n, m,front = 0, rear = 0,fresh = 0,time = 0,i, j,r, c,nr, nc,dr[] = {-1, 1, 0,
0},dc[] = {0, 0, -1, 1};
    printf("Enter rows and columns: ");
    scanf("%d %d", &n, &m);

    printf("Enter matrix:\n");

    for(i = 0; i < n; i++)
    {
```

```
        for(j = 0; j < m; j++)
        {
            scanf("%d", &a[i][j]);

            if(a[i][j] == 2)
            {
                queue[rear][0] = i;
                queue[rear][1] = j;
                rear++;
            }

            if(a[i][j] == 1)
                fresh++;
        }
    }
    while(front < rear && fresh > 0)
    {
        int size = rear - front;

        for(i = 0; i < size; i++)
        {
            r = queue[front][0];
            c = queue[front][1];
            front++;

            for(j = 0; j < 4; j++)
            {
                nr = r + dr[j];
                nc = c + dc[j];

                if(nr >= 0 && nr < n &&
                   nc >= 0 && nc < m &&
                   a[nr][nc] == 1)
                {
                    a[nr][nc] = 2;
                    fresh--;

                    queue[rear][0] = nr;
                    queue[rear][1] = nc;
                    rear++;
                }
            }
        }

        time++;
    }
```

```c
    if(fresh == 0)
        printf("Minimum time = %d\n", time);
    else
        printf("Cannot rot all oranges = -1\n");

    return 0;
}
```

Output:

